TEE Private Intent Network: Implementation Plan

Nov 6, 2026 · @Apurva

1. Scope and goal

Build one complete flow on one EVM network: deposit ETH, get a confidential balance, submit a conditional swap, have it executed from pooled inventory, and withdraw tokens to an existing wallet. One real TEE node holds every key. This follows section 9 of your brief.

In scope

- One attested TEE node that generates all EOA keys internally
- One asset pair and one router (Uniswap)
- Fungible claims against pooled inventory, not per-user accounts
- Deposit and withdrawal by ordinary transfer, using the user's existing wallet
- One conditional intent type: swap when a price condition is met
- A frontend for deposit, balance, intent status and withdrawal

Out of scope (say so in the demo)

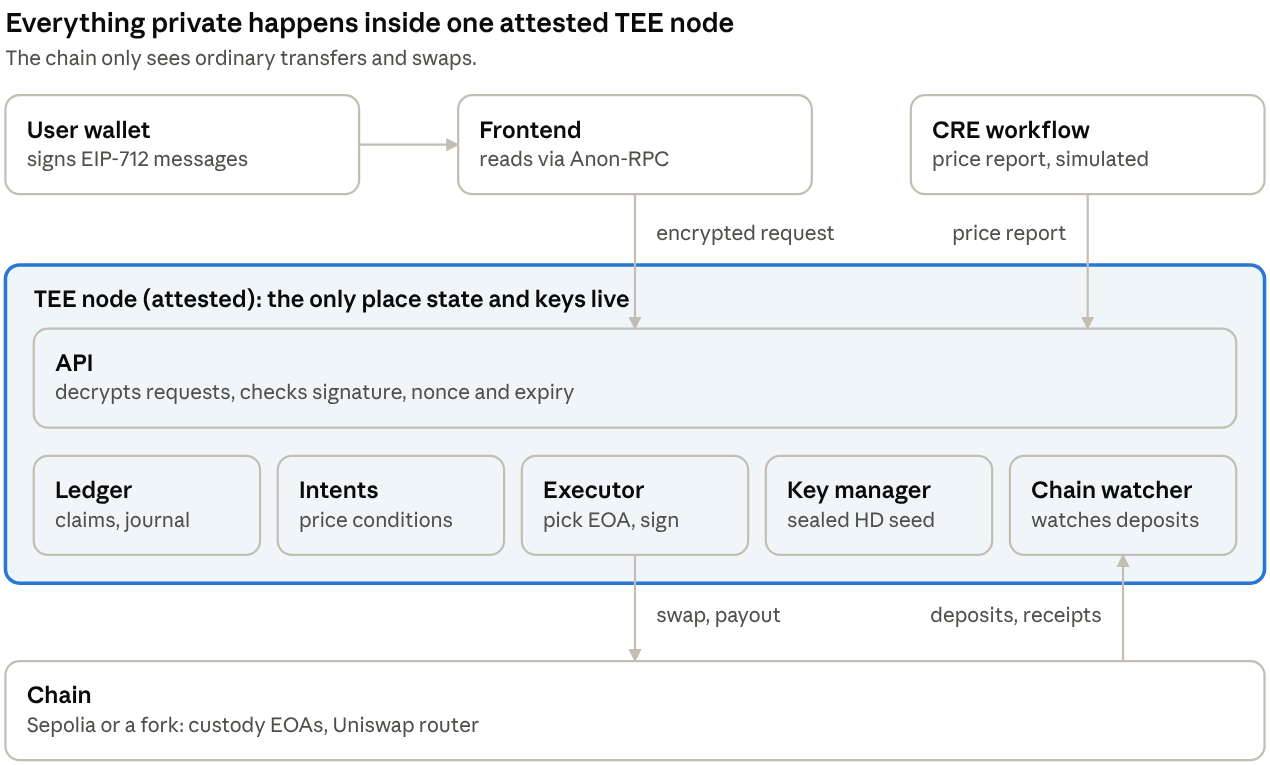
- Key handover, multi-node or threshold custody, slashing
- General DeFi execution, ZK proofs, anonymity guarantees

Claim to make: private ownership coordination and intent execution with no mandatory privacy-contract call. Never claim that usage is undetectable.

Definition of done: a judge can watch two users deposit, one swap run from an EOA that neither user funded, and a payout land in a normal wallet. Replayed requests, unauthorized destinations and concurrent overspends must be rejected live.

2. Architecture

One attested TEE node holds every key and the whole ledger. Everything outside it, the wallet, the frontend, the CRE workflow and the chain, only sees signed requests, price reports or ordinary transactions.



architecture · TEE node, wallet, frontend, CRE, chain

Users and the CRE workflow only talk to the API. The chain sees ordinary transfers from custody EOAs and swaps through the Uniswap router.

Module	Responsibility
API	Decrypt requests, verify signature, nonce and expiry, hand off to the ledger
Ledger	Balances, reservations, journal, reconcile check
Intents	Store intents and evaluate price conditions on each report
Executor	Choose an inventory EOA, build the swap or transfer, sign, broadcast, settle
Key manager	Generate and seal the HD seed, derive EOAs, sign only for approved operations
Chain watcher	Track deposits, confirmations and receipts

3. Technical requirements

The stack is TypeScript end to end, with Foundry for chain tooling, so four people can work in parallel and no custom contracts are needed. Items marked *verify* depend on provider docs I have not checked.

Layer	Choice	Notes
TEE runtime	Pick one in the first 3 hours: EigenCompute (EigenCloud), Phala dstack (Intel TDX), or AWS Nitro Enclaves	*Verify* current access, pricing and attestation format. Fallback: same container, clearly labelled TEE-simulated
Enclave service	TypeScript, Node 20+, Fastify, zod, viem	One process; all ledger writes go through a single serialized queue
Key management	Seed generated inside the enclave, EOAs derived by HD path, seed sealed to the enclave	No key export endpoint. Never log keys or addresses of custody EOAs
Ledger storage	In-memory state plus an append-only, hash-chained journal, encrypted before it touches disk	PostgreSQL only for non-secret metadata, if at all
Chain	Sepolia, or an Anvil mainnet fork if Uniswap liquidity is missing there	Decide in phase 0 (see section 10)
DEX	Uniswap: Trading API for quotes, Universal Router or SwapRouter for execution	One pair, for example WETH to a test USDC. Confirm a pool with liquidity on the chosen network
Client signing	EIP-712 through wagmi and viem, signed by the user's existing wallet	No custom wallet. Auth key is separate from custody keys
Frontend	Next.js, wagmi, RainbowKit, Anon-RPC browser harness for chain reads	No third-party analytics
Trigger	Chainlink CRE workflow, run through the CRE CLI simulation	*Verify* CLI setup and access early
Packaging	Docker image, pinned dependencies, reproducible build	Judges and attestation both want a build they can rebuild

Layer	Choice	Notes
Tests	Foundry and Anvil for chain, Vitest for ledger and API	Reconcile check runs after every test

Accounts and funds to arrange before hour 0

- TEE provider account and a deploy that returns an attestation quote
- RPC provider key and testnet ETH for 4 to 6 custody EOAs
- Uniswap developer platform key
- WalletConnect project ID
- Test token liquidity, or a funded fork

4. Data model and accounting invariants

Every asset the pool holds must be explainable as a claim, per token. The ledger lives only inside the enclave; nothing here is public.

Entity	Key fields	States
Owner	wallet address, last nonce	active
Balance	owner, token, available, reserved	one row per owner and token
DepositSession	session ID, assigned deposit EOA, owner, expiry	open, credited, expired
CustodyEOA	index, address, role, last used	deposit, inventory
Intent	owner, tokenIn, tokenOut, amountIn, price condition, max slippage, expiry	pending, executing, done, failed, cancelled
Withdrawal	owner, token, amount, destination, tx hash	locked, sent, confirmed, failed
Journal entry	sequence number, event, previous hash	append-only

Invariants to enforce in code and assert in tests

1. Per token, on-chain balances across all custody EOAs (minus a gas reserve) are at least the sum of every owner's available, reserved and pending-withdrawal amounts.

2. A surplus in one token never covers a shortfall in another.
3. Available and reserved are never negative. Funds are reserved before an intent or withdrawal can proceed.
4. Each owner nonce is used once. Each on-chain deposit is credited once.
5. Every broadcast transaction maps to exactly one intent or withdrawal, and its hash is journaled before broadcast so a restart cannot execute it twice.

Fees: charge a fixed fee per intent and per withdrawal in the input token. Book gas spent by custody EOAs against a separate fee ledger so invariant 1 still holds.

Concurrency rule: one serialized queue handles every ledger mutation. It is the simplest defence against double spends, and fast enough for a demo.

5. Core flows

Three flows make the whole product. Each ends in a ledger state change that the reconcile check can verify.

Deposit

1. The user signs a DepositRequest (owner, token, chain, nonce, expiry) with their existing wallet and sends it to the enclave.
2. The enclave verifies the signature and nonce, assigns a fresh unused EOA exclusively to this session, and returns its address and expiry.
3. The user sends ETH from the same wallet to that address with an ordinary transfer.
4. The chain watcher matches sender and recipient to the open session. After N confirmations it credits the owner's available balance and tags the EOA as inventory.
5. A transfer from the wrong sender, or after expiry, is not credited and is logged for manual refund (out of scope).

Conditional swap

1. The user signs an Intent: tokenIn, tokenOut, amountIn, a price condition, max slippage, expiry, nonce.
2. The enclave checks that available covers amountIn, moves it to reserved, and stores the intent.
3. On each price report the enclave evaluates pending intents. A report comes from the CRE workflow, and the enclave also checks it against its own Uniswap quote.

4. When a condition is met, the executor picks an inventory EOA that holds enough tokenIn and is not the owner's deposit EOA. It builds the swap with a minimum output from the slippage setting, signs inside the enclave, and broadcasts.
5. On confirmation it settles from the receipt: reserved tokenIn is consumed and the owner's tokenOut balance rises by the actual output. On revert or expiry it releases the reservation.

Withdrawal

1. The user signs a Withdrawal binding owner, token, amount, destination, chain, expiry and nonce.
2. The enclave verifies it, locks the amount, and picks an inventory EOA with enough of that token, avoiding the owner's own deposit EOA.
3. It sends a normal transfer to the signed destination and finalizes after confirmation.
4. If no single EOA holds enough, the request is rejected and the lock is released. There is no consolidation step in the MVP.

Inventory seeding: before the demo, fund 4 to 6 EOAs with ETH and the output token so withdrawals never depend on a lucky deposit. Say so openly in the demo.

6. API and signing spec

Every state-changing request is an EIP-712 message signed by the owner's wallet and encrypted to the enclave. This is what makes replays, tampered destinations and double spends testable.

EIP-712 domain

- name: PrivateIntentNetwork, version: 1, chainId of the execution network
- No verifyingContract exists, since there is no privacy contract. Use the salt field, set to a hash of the enclave public key, so signatures only verify against this enclave.

Message types: DepositRequest, Intent, Withdrawal, BalanceQuery. Each carries owner, a nonce and an expiry. Intent and Withdrawal also carry the full amounts, tokens and, for withdrawals, the destination.

Replay protection (the application must build this)

- Strictly increasing nonce per owner, checked inside the serialized ledger queue.
- Short expiry on every request; reject anything expired or dated in the future.
- Reject any request whose chainId or salt does not match.

Transport: TLS may terminate outside the enclave on some platforms, so encrypt the request body to the enclave's attested public key at the application layer. The frontend shows the attestation measurement and a link to the reproducible build. Full in-browser quote verification is a stretch goal; a CLI check is the minimum.

Method and path	Auth	Purpose
GET /attestation	none	Quote, enclave public key, build hash
POST /deposit-session	signed DepositRequest	Assign a deposit EOA
POST /balance	signed BalanceQuery	Available, reserved, pending per token
POST /intents	signed Intent	Create and reserve
POST /intents/status	signed status query	Status of the owner's intents
POST /withdrawals	signed Withdrawal	Lock, send, finalize
POST /oracle/price	CRE workflow signature	Price report for condition checks
GET /health/reconcile	none	Aggregate invariant result only, no per-user data

Never return custody EOA addresses except the one assigned to a deposit session. Keep logs free of owner addresses and amounts.

7. Prize integrations

Three prizes, each wired into the real flow so none is a bolt-on. Requirements are from the ETHGlobal Mumbai prizes page.

Prize	What to build	Evidence to ship	Watch out
Uniswap Foundation, Build with Uniswap ($10,000 pool)	Swap execution through the Uniswap API and router from custody EOAs	Public open-source repo, `FEEDBACK.md`, completed Uniswap feedback form with the file link, README pointing to the integration code lines	Submissions without the feedback file are audited before winners are set
Ethereum Foundation, Anon-RPC ($5,000), with CROPS ($3,500) as backup	Frontend sends its chain reads through Anon-RPC by default	Demo showing reads going through it, plus a list of any traffic left out and why	Judges score how much traffic uses it. EF places a project in at most one EF prize, and CROPS needs a README section titled CROPS plus an open-source licence
Chainlink, Best workflow with CRE ($1,500)	A CRE workflow reads an external price API and an onchain source, then posts a signed price report to `/oracle/price`	Successful CRE CLI simulation shown in the demo (deployment is optional)	It must be meaningfully used. The enclave still cross-checks the price and enforces a minimum output, so a bad report cannot drain funds

Repo hygiene for all three: public repo from the first commit, a licence file (MIT or Apache-2.0), steady commit history through the event, and a README that covers setup, architecture, the trust assumptions and the known leaks.

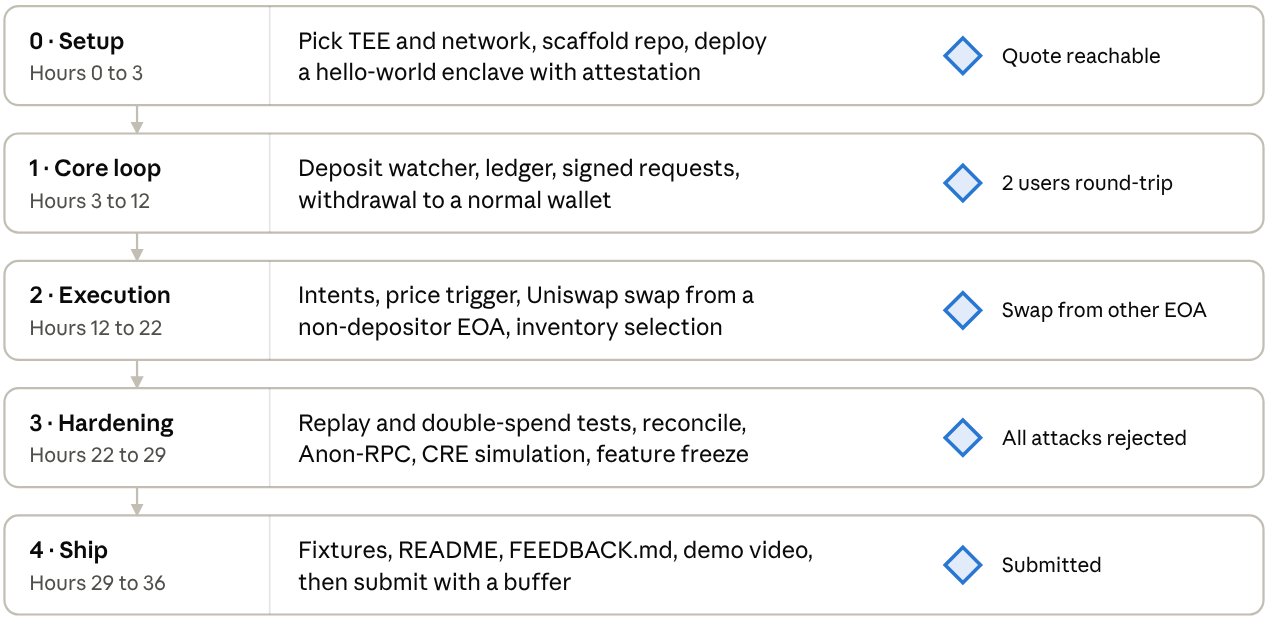
Trade-off to state: the enclave's own RPC reads and the CRE workflow both see something. Anon-RPC covers the browser only, so list the enclave's reads as a leak instead of hiding it.

8. Build order and team split

The plan assumes a 36-hour event and four people. Shift the hour marks if your clock differs. Each phase ends at a gate; do not start the next phase's features until the gate passes.

Each phase ends at a gate that must pass before the next begins

36-hour build, four people. A diamond marks the gate.



build roadmap · 5 phases, 5 gates

If a gate slips, cut scope from the next phase, never the rejection tests.

Person	Workstream	Owns	Done when
A	TEE and custody	Enclave deploy, attestation, HD key derivation, signing guard, encrypted transport	Quote verifies and a derived EOA signs a test transaction
B	Accounting and API	Ledger, journal, nonces, EIP-712 verification, invariants	Concurrency and replay tests pass
C	Chain and execution	Deposit watcher, confirmations, inventory selection, Uniswap swap, payouts	A swap and a payout settle from a non-depositor EOA

Person	Workstream	Owns	Done when
D	Frontend and integrations	Next.js app, Anon-RPC, CRE workflow, README, `FEEDBACK.md`, demo video	The full flow runs from the UI

Everyone writes tests for their own part and joins the demo rehearsal.

9. Test plan and demo script

The rejection tests are the proof that the ledger holds. Run the reconcile check after every test, and show the attack tests live.

Test	Expected result
Resend a signed withdrawal	Rejected: nonce already used
Change the destination after signing	Rejected: signature does not verify
Submit an expired request	Rejected
Two withdrawals of 80% of a balance sent at once	Exactly one succeeds
Intent and withdrawal race on the same balance	Only one gets the reservation
Withdraw a token the user does not hold while the pool has surplus ETH	Rejected: no cross-token cover
Withdraw more than any single EOA holds	Clean rejection, lock released
Kill the enclave after broadcast, before settlement	On restart the state reconciles and nothing executes twice
Deposit with fewer than N confirmations	Not credited yet
Deposit from the wrong sender or after expiry	Not credited
Price report outside the enclave's own quote tolerance	Intent does not execute

Demo script (about 3 minutes)

1. Show the attestation page and state the trust model in one sentence.
2. Alice and Bob deposit from different wallets. The explorer shows two plain transfers to two EOAs.
3. The app shows each user's confidential balance.
4. Alice signs a conditional swap. The CRE simulation posts the price that triggers it.
5. The explorer shows the swap sent from an EOA that neither user funded.
6. Alice withdraws to a fresh wallet and receives a normal transfer.
7. Run the replay, wrong-destination and double-spend attacks. All are rejected.
8. Close on the known-leaks slide: amounts and timing, the visible payout edge, gas funding, and the enclave's own RPC reads.

10. Risks, fallbacks and decisions

Decide the six items below before hour 0. The biggest schedule risk is TEE setup, so it is time-boxed to the first 3 hours.

Risk	Fallback
TEE setup eats the schedule	Run the same container outside a TEE, label it TEE-simulated in the UI and README, and make no confidentiality claim
No Uniswap liquidity on the chosen testnet	Run on an Anvil mainnet fork and call it a fork-based demonstration, as your brief allows
Inventory shortage during the demo	Pre-seed 4 to 6 EOAs and reject clean when a request cannot be filled
CRE access or setup problems	Let the enclave poll its own Uniswap quote and treat CRE as optional
Anon-RPC only covers the browser	Route frontend reads only and list the enclave's reads as a leak
Tiny anonymity set with 2 to 3 demo users	Claim no anonymity. Use equal deposit sizes and varied delays, and say the set is small
State loss on a single node	Journal only, no backup in the MVP, documented as a limit

Decisions to make now

1. **TEE platform.** EigenCompute, Phala dstack or Nitro Enclaves; whichever returns an attestation quote first.
2. **Network.** Sepolia, or a fork. None of Whitechain's four tracks fits as written: each needs a USDT payment settled on Whitechain Sepolia in the main flow, and they target payments, not swaps.
3. **Relationship to hide.** I recommend deposit to withdrawal and deposit to swap. Swap to withdrawal can still leak through amounts and timing.
4. **Withdrawal credential.** I recommend the depositor's wallet signing EIP-712, with recovery meaning the same wallet. The custody keys are never involved.
5. **Initial inventory.** Operator-funded seed EOAs, disclosed in the demo.
6. **Open-source licence.** MIT or Apache-2.0 from the first commit.

Stretch, only if the phase 3 gate passes: a natural-language box that turns "swap 0.1 ETH to USDC if ETH hits X" into a structured Intent for the user to review and sign. The model never touches keys or the enclave.